Node.js Architecture & Core Concepts

Full Stack & AI Developer Syllabus — Day 28 Detailed Guide

1. What is Node.js? (Architecture & V8 Engine)

Node.js is an open-source, cross-platform JavaScript runtime environment built on Google Chrome's **V8 JavaScript engine**. It enables developers to execute JavaScript on the server side, powering high-performance REST APIs, backend microservices, and command-line tools.

Component	Role & Responsibility
V8 Engine	C++ engine that compiles JavaScript source code directly into optimized native machine code using Just-In-Time (JIT) compilation.
libuv	Multi-platform C library providing the event loop, thread pool, and non-blocking asynchronous I/O abstractions for filesystem and network ops.
Non-Blocking I/O	Single-threaded execution loop delegates heavy I/O tasks to libuv background threads, preventing server process blockage.

2. Installing Node.js 20 LTS, REPL & Script Execution

Node.js 20 LTS provides enterprise stability, native permissions management, and updated V8 features. Developers interact with Node using terminal commands and the REPL shell.

```
// 1. Verify Installation in Terminal
node -v  # Output: v20.18.0

// 2. Interactive REPL (Read-Eval-Print Loop)
$ node
> 5 + 10
15
> const name = 'Node.js 20 LTS';
> name.toUpperCase();
'NODE.JS 20 LTS'
> .exit

// 3. Executing a .js file
$ node server.js
```

3. Global Objects in Node.js

Node.js provides global identifiers available across all modules without explicit imports. Unlike browser JavaScript where window is global, Node uses global and exposes system runtime context.

```
/** global-demo.js */
console.log('Process ID (PID):', process.pid);
console.log('Node Version:', process.version);
console.log('Platform:', process.platform);
console.log('Current Working Dir:', process.cwd());
console.log('__dirname:', __dirname);   // Absolute directory path
console.log('__filename:', __filename); // Absolute file path
```

4. CommonJS Module System (require & module.exports)

CommonJS is the standard synchronous module system in Node.js. Code encapsulation is achieved by assigning exported features to module.exports and importing them using require().

```
// formatter.js (Module Export)
function formatCurrency(amount) {
  return `$${amount.toFixed(2)}`;
}
module.exports = { formatCurrency, APP_VERSION: 'v2.0' };

// app.js (Module Import)
const { formatCurrency, APP_VERSION } = require('./formatter');
console.log(`Version ${APP_VERSION}:`, formatCurrency(1250.5)); // $1250.50
```

5. Built-in Modules: path, os, and fs

Node.js ships with core modules out of the box. Key system modules include path (cross-platform file paths), os (hardware info), and fs (filesystem operations).

```
const path = require('path');
const os = require('os');
const fs = require('fs');

// 1. Path manipulation
const logPath = path.join(__dirname, 'logs', 'app.log');
const ext = path.extname('report.pdf'); // '.pdf'

// 2. OS metrics
console.log('OS Platform:', os.platform());
console.log('CPU Cores:', os.cpus().length);
console.log('Free Mem (GB):', (os.freemem() / (1024 ** 3)).toFixed(2));

// 3. File System (Asynchronous operations)
fs.mkdir(path.join(__dirname, 'data'), { recursive: true }, (err) => {
  if (err) throw err;
  fs.writeFile(path.join(__dirname, 'data/test.txt'), 'Hello Node.js!', 'utf-8', (err) => {
    if (err) throw err;
    console.log('File created and written successfully!');
  });
});
```

6. Industrial Best Practice Summary

> **Key Takeaways for Node.js 20 Backend Engineers:**
> 1. Always use path.join() or path.resolve() instead of manual string concatenation to avoid cross-OS path bugs.
> 2. Avoid synchronous fs.*Sync methods in HTTP server request handlers as they block the event loop for all users.
> 3. Keep module dependencies organized with CommonJS or modern ES Modules and leverage process.env for configuration.